Software Requirements Specification (SRS)

1. Purpose

This SRS records the behavior implemented by the reviewed repository and distinguishes it from recommended production requirements. “Implemented” means a matching code path exists; it does not mean the behavior is fully tested or production-safe.

2. Actors and authorization

Actor	Public operations	Authenticated operations
Guest	Browse/search temples, temple details, and slots; register/login	None
USER	Same as guest	Profile, review, booking hold/confirmation/history/cancellation, donation/history
ORGANIZER	Same as guest	Profile, slot create/update/delete, all booking and donation records
ADMIN	Same as guest	All organizer functions plus temple, event, maintenance, user, and analytics administration

API middleware enforces most protected operations. Frontend dashboard URLs themselves are not guarded, but unauthorized API calls are rejected if a valid privileged JWT is absent.

3. Functional requirements

ID	Requirement	Status
FR-01	Register a user with name, email, password, optional phone/address	Implemented
FR-02	Authenticate by email/password and issue a 30-day JWT	Implemented
FR-03	View and update the authenticated profile	Implemented
FR-04	Search temples by partial, case-insensitive name/location	Implemented
FR-05	View temple media, hours, amenities, events, ratings, and reviews	Implemented
FR-06	Add or update one review per authenticated user per temple	Implemented
FR-07	List slots, optionally filtered by temple and calendar date	Implemented

ID	Requirement	Status
FR-08	Create a five-minute pending booking hold and deduct seats	Implemented
FR-09	Calculate booking total as devotee count × stored slot price	Implemented
FR-10	Confirm a pending booking after simulated client-side card checks	Implemented; no real gateway
FR-11	Expire pending holds and restore capacity	Implemented lazily when booking/slot APIs run
FR-12	Display authenticated user booking history and ticket details	Implemented
FR-13	Cancel a booking and restore capacity	Partially implemented; pending cancellations do not restore seats immediately
FR-14	Record a donation against a temple and generate a receipt	Implemented; payment simulated
FR-15	Display authenticated user donation history	Implemented
FR-16	Allow organizers/admins to manage slots	Implemented
FR-17	Allow organizers/admins to view booking/donation registries	Implemented
FR-18	Allow admins to create/update/delete temples through API	Implemented; UI creates/deletes only
FR-19	Allow admins to add/delete events through API	Implemented; UI adds events
FR-20	Allow admins to add maintenance requests and update status	Implemented
FR-21	Allow admins to list/filter/update/delete users	Implemented in API; UI lists/filters/deletes
FR-22	Provide analytics for users, organizers, temples, booked revenue, donations, popularity, and age groups	Implemented
FR-23	Seed initial temples and slots when the temple collection is empty	Implemented

4. Business rules

- Email is unique and normalized to lowercase on persistence.
- Passwords are hashed with bcrypt using a generated salt.
- Roles are one of `USER`, `ORGANIZER`, or `ADMIN`.
- A booking requires at least one devotee with name, age, and gender.
- Gender is `Male`, `Female`, or `Other`.
- Booking status is `Pending`, `Booked`, `Cancelled`, or `Expired`.
- A hold expires five minutes after creation.
- Slot price is server-authoritative; the client does not submit a booking total.
- A review rating must be between 1 and 5 according to the schema.
- Donation amount must be at least 1 according to the schema.
- Donation purpose must match one of five schema values.
- Maintenance status is `Pending`, `In Progress`, or `Completed`.

5. Non-functional requirements

Implemented properties

- Responsive single-page interface.
- JSON REST API and consistent top-level `success` indicator.
- Password hashing and JWT-based authentication.
- Mongoose schema constraints and timestamps.
- API returns `503` while MongoDB is disconnected.
- Central fallback 404 and generic error middleware.

Required before production

ID	Requirement	Acceptance target
NFR-01	Security	Server assigns public registrations `USER`; secrets are mandatory environment values; rate limits and validation enabled
NFR-02	Consistency	Concurrent booking attempts cannot oversell; booking/seat updates are atomic
NFR-03	Availability	Expiration runs independently of user traffic and database reconnection is handled
NFR-04	Performance	Define and load-test p95 response targets under expected concurrency
NFR-05	Accessibility	Keyboard navigation, labels, focus management, and WCAG 2.1 AA review
NFR-06	Observability	Structured logs, request IDs, metrics, traces, alerts, and health/readiness checks
NFR-07	Recoverability	Automated backups and verified restore procedure
NFR-08	Quality	Unit, integration, and E2E suites run in CI with agreed coverage gates

6. Constraints and assumptions

- Node.js and npm are required for both modules.
- A reachable MongoDB instance is required for all `/api` requests.
- The frontend currently assumes the API is at `http://localhost:5000/api`.
- Seed dates are calculated relative to the first database startup.
- Media uses a mixture of client-local paths and remote video URLs.
- The repository contains `server/node_modules`, which should normally be excluded and recreated from the lockfile.